```
import argparse
import datetime
# Define main parser
parser = argparse.ArgumentParser(description='Task Manager CLI')
subparsers = parser.add_subparsers(dest='command')
# Add subcommands
add_parser = subparsers.add_parser('add', help='Add a task')
add_parser.add_argument('task', type=str, help='Task description')
add_parser.add_argument('--due', type=str, help='Due date (YYYY-MM-DD)')
list_parser = subparsers.add_parser('list', help='List tasks')
list_parser.add_argument('--status', type=str, choices=['pending', 'done'], help='Filter by
status')
list_parser.add_argument('--due', type=str, help='Filter by due date')
done_parser = subparsers.add_parser('done', help='Mark task as done')
done_parser.add_argument('task_id', type=int, help='Task ID')
# Basic task storage (in-memory for now)
tasks = []
def add_task(task, due=None):
    tasks.append({'id': len(tasks)+1, 'task': task, 'due': due, 'status': 'pending'})
    print(f"Task added (ID: {len(tasks)})")
def list_tasks(status=None, due=None):
    filtered = tasks
    if status:
        filtered = [t for t in tasks if t['status'] == status]
    if due:
        filtered = [t for t in tasks if t['due'] == due]
    for t in filtered:
        print(f"{t['id']}: {t['task']} (Due: {t['due']}, Status: {t['status']})")
def mark_done(task_id):
```

```
    for t in tasks:
        if t['id'] == task_id:
            t['status'] = 'done'
            print(f"Task {task_id} marked done")
            return
    print("Task not found")
def main():
    args = parser.parse_args()
    if args.command == 'add':
        add_task(args.task, args.due)
    elif args.command == 'list':
        list_tasks(args.status, args.due)
    elif args.command == 'done':
        mark_done(args.task_id)
if __name__ == '__main__':
    main()
#Here's a breakdown:
- I used`argparse` for subcommands like `add`, `list`, `done`, `filter`.
- Then I Included filters by status (e.g., pending, done) or due date.
- Write at least three tests with `pytest` for core functionality.
- Publish to a repo  showing install and usage examples.
Smallest version first, then iterate.#
```